COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPOE21S1
DATE PERFORMED:	August 4 2026	**DATE SUBMITTED:**	August 11 2026
NAME:	Adrian Walter D. Santos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO.5 **QUEUE**

1. PROCEDURE OUTPUT

ILO A

Code

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main(){
    std::queue<char> q;

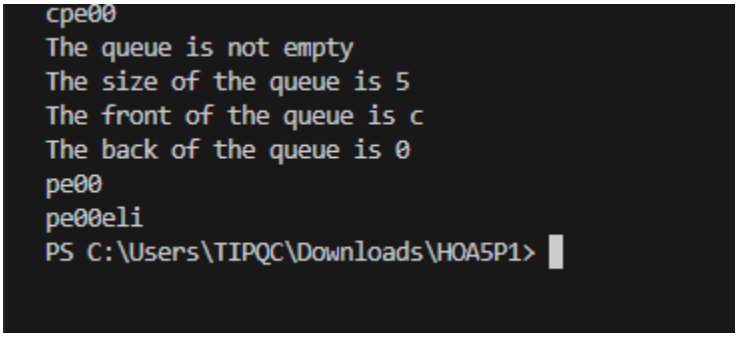
    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('0');

    //display
    display(q);

    //empty
    if (q.empty()){
        std::cout<<"The queue is empty \n";

    } else {
        std::cout<<"The queue is not empty \n";

    };
    //size
    std::cout<<"The size of the queue is "<<size(q) <<"\n";
    //front
    std::cout<<"The front of the queue is "<<q.front()<<"\n";
    //back
    std::cout<<"The back of the queue is "<<q.back()<<"\n";
    //pop
    q.pop();
    //display
    display(q);
    //push another element
    q.push('e');
    q.push('1');
    q.push('i');
```

	
Output	
Analysis	When dealing with a queue the first in first out principle is applied. This means that whatever value is pushed in first is the first one that is going to be popped. The code demonstrates how to create, inspect, and modify a standard character queue step by step. It begins by filling a queue with letters, printing its contents, and checking basic details like its overall size, whether it is empty, and which elements sit at the very front and back. Finally, it removes the leading character, adds new characters to the end, and uses a clever helper function that safely prints the queue by looping through a temporary duplicate so the original data remains intact.

ILO B

LINKED LIST IMPLEMENTATION

Code

```cpp
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue(&front, &back, 'J');
    std::cout<< front->data << " "<<back->data<<std::endl;

    enqueue(&front, &back, 'J');
    std::cout<< front->data << " "<<back->data<<std::endl;
    enqueue(&front, &back, 'I');
    std::cout<< front->data << " "<<back->data<<std::endl;
    enqueue(&front, &back, 'M');
    std::cout<< front->data << " "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<< front->data << " "<<back->data<<std::endl;

    return 0;
}
```

Main.cpp

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
    public:
        T data;
        qNode *next;
};

//Creating a new node
template <typename T>
qNode<T>* new_Node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template<typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    //create a new node
    qNode<T>* newNode = new_Node(newData);

    //inserting to an empty queue
    if((*frontPtr) == nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    //inserting an item into a non empty queue
    // point the backptr next to the newNode
    (*backPtr)->next = newNode;
    (*backPtr)= newNode;
}

```

```
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a temporary node to store the node to be deleted
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);

    //check if the queue is only q qNode

    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    //deleting of the node
    (*frontPtr)= deleteNode->next;
    delete deleteNode;


}
//display all elements in the list

template <typename T>
void display(qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a copy of the queue
    std::queue<char> c = r;

    while(!c.empty()){
        std::cout<<c.front();
        c.pop();
    }
    std::cout<<"\n";
}
```

```
    //return the front
#endif
```

qeueull.h

Output

```
J J
J J
J I
J M
J M
PS C:\Users\TIPQC\Downloads\HOA5P1>
```

Analysis

This program demonstrates how a custom linked-list queue stores and manages character data

	through direct node links. In main(), character items ('J', 'J', 'I', 'M') are added step-by-step using an enqueue function, printing the current front and back values after each addition. Finally, calling dequeue removes the oldest item ('J') from the front of the line, leaving the second 'J' at the front while 'M' remains at the back.

ARRAY IMPLEMENTATION

Code

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

//array based circular queue
template <typename T>
class queueArr {
    private:
        //create a pointer to an array of type T
        T* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        //prototyppes
        queueArr(size_t capacity);
        bool Empty();
        bool Full();

        size_t Size();
        void Clear();

        T Front();
        T Back();
        void Enqueue(T newData);
        void Dequeue();

        ~queueArr();

        queueArr(const queueArr& other);
        queueArr& operator=(const queueArr& other);
};
```

```cpp
//constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
    std::cout<<"Constructor class was called"<<std::endl;
}

//destructor
template <typename T>
queueArr<T>::~queueArr() {
    delete[] q_array;
}

//empty()
template <typename T>
bool queueArr<T>::Empty() {
    return q_size == 0;
}

//full()
template <typename T>
bool queueArr<T>::Full() {
    return q_size == 0;
}

```

```cpp
template <typename T>
void queueArr<T>::Clear() {
    q_size = 0;
    q_front = 0;
    q_back = -1;
}
template <typename T>
T queueArr<T>::Front(){
    //Check and display if the queue is empty
    if (Empty()) {
        std::cout << "Queue is empty. " << std::endl;
        return T{}; // Return default value of T
    }
    //return the front item
    return q_array[q_front];

}

template <typename T>
T queueArr<T>::Back() {
    //Check and display if the queue is empty
    if (Empty()) {
        std::cout << "Queue is empty. " << std::endl;
        return T{}; // Return default value of T
    }
    //return the back item
    return q_array[q_back];
}
```

```cpp
template <typename T>
size_t queueArr<T>::Size() {

    return q_size;
}


template <typename T>
void queueArr<T>::Enqueue(T Value) {
    //Check and display if the queue is full
    if (Full()) {
        std::cout << "Queue is full. " << std::endl;
        return;
    }
    //increment the back index in a circular manner
    q_back = (q_back + 1) % q_capacity;
    //add the new value to the back of the queue
    q_array[q_back] = Value;
    //increment the size of the queue
    q_size++;
}

template <typename T>
void queueArr<T>::Dequeue() {
    //Check and display if the queue is empty
    if (Empty()) {
        std::cout << "Queue is empty. " << std::endl;
        return;
    }
    //increment the front index in a circular manner
    q_front = (q_front + 1) % q_capacity;
    //decrement the size of the queue
    q_size--;
}
```

```
131  // Copy Constructor:
132  template <typename T>
133  queueArr<T>::queueArr(const queueArr& other) {
134      q_capacity = other.q_capacity;
135      q_size = other.q_size;
136      q_front = other.q_front;
137      q_back = other.q_back;
138      q_array = new T[q_capacity];
139      for (size_t i = 0; i < q_capacity; i++) {
140          q_array[i] = other.q_array[i];
141      }
142  }
143
144  // Copy Assignment Operator
145  template <typename T>
146  queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
147      if (this == &other) {
148          return *this; // guard against q = q;
149      }
150
151      delete [] q_array; // free our old array first
152
153      q_capacity = other.q_capacity;
154      q_size = other.q_size;
155      q_front = other.q_front;
156      q_back = other.q_back;
157      q_array = new T[q_capacity];
158      for (size_t i = 0; i < q_capacity; i++) {
159          q_array[i] = other.q_array[i];
160      }
161
162      return *this;
163  }
164  #endif
```

Header files

```cpp
#include <iostream>
#include "queueArr.h"
#include "queueArr.h"

int main() {

    //create an object
    queueArr<int> q(5);
    q.Enqueue(1);
    std::cout << "Front: " << q.Front() << std::endl;
    q.Enqueue(2);
    std::cout << "Back: " << q.Back() << std::endl;
    q.Enqueue(3);
    q.Enqueue(4);
    q.Dequeue();
    std::cout << "Front: " << q.Front() << std::endl;
    q.Size();
    std::cout << "Size: " << q.Size() << std::endl;

    // Testing Copy Constructor
    queueArr<int> q2 = q;
    q2.Dequeue(); // Modifying q2 to show it doesn't affect q

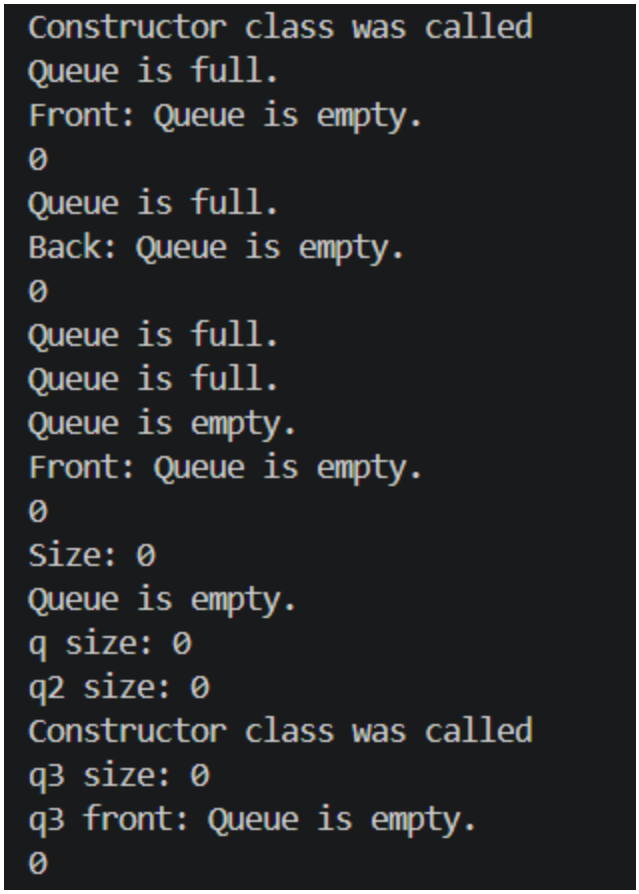
    std::cout << "q size: " << q.Size() << std::endl;   // Expected: 3
    std::cout << "q2 size: " << q2.Size() << std::endl; // Expected: 2

    // Testing Copy Assignment Operator
    queueArr<int> q3(2);
    q3 = q;

    std::cout << "q3 size: " << q3.Size() << std::endl;   // Expected: 3
    std::cout << "q3 front: " << q3.Front() << std::endl; // Expected: 1


    return 0;
}
```

Driver function

Output	
Analysis	This code creates a custom queue class called queueArr that uses a fixed-size array to store and manage numbers. It uses a circular setup, which allows the array to reuse empty slots as items are added and removed. In main(), the program creates an initial queue q, adds several numbers (1, 2, 3, 4), removes the first element, and prints the updated front, back, and size values. It then creates two duplicates of the queue using copy operations to test how independent copies behave. Ultimately, modifying the copied queues (q2 and q3) leaves the original queue (q) completely untouched, proving that each queue safely owns its own data.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

```cpp
#include <iostream>
#include <string>
#include "queueArr.h"

// Represents a single print task
struct Job {
    int id;
    std::string userName;
    int pages;

    // Default constructor needed for array initialization in template
    Job() {
        id = 0;
        userName = "";
        pages = 0;
    }

    Job(int jId, std::string name, int numPages) {
        id = jId;
        userName = name;
        pages = numPages;
    }

    void printInfo() const {
        std::cout << "Job #" << id << " (" << userName << ", " << pages << " pages)";
    }
};
```

Struct Job that includes ID ,user name, and number of pages

```cpp
// Manages printer jobs using the custom circular queue
class Printer {
private:
    queueArr<Job> jobQueue;

public:
    Printer(int maxCapacity) : jobQueue(maxCapacity) {}

    // Add a job to the queue
    void submitJob(Job newJob) {
        if (jobQueue.Full()) {
            std::cout << "[Error] Printer queue is full! Couldn't add ";
            newJob.printInfo();
            std::cout << std::endl;
            return;
        }

        jobQueue.Enqueue(newJob);
        std::cout << "Added: ";
        newJob.printInfo();
        std::cout << " to the queue." << std::endl;
    }
```

```
    // Process single job at the front
    void printNext() {
        if (jobQueue.Empty()) {
            std::cout << "No jobs in the queue to print." << std::endl;
            return;
        }

        Job current = jobQueue.Front();
        std::cout << "\nCurrently printing: ";
        current.printInfo();
        std::cout << "..." << std::endl;

        jobQueue.Dequeue();
        std::cout << "Finished printing Job #" << current.id << std::endl;
    }

    // Run through remaining jobs
    void printAll() {
        std::cout << "\n--- Starting printer run ---" << std::endl;
        while (!jobQueue.Empty()) {
            printNext();
        }
        std::cout << "--- Done! Queue is empty. ---" << std::endl;
    }
};
```

Printer Class

```cpp
int main() {
    // Shared printer with max 5 jobs
    Printer officePrinter(5);

    std::cout << "=== Submitting Initial Jobs ===" << std::endl;
    officePrinter.submitJob(Job(1, "Alice", 12));
    officePrinter.submitJob(Job(2, "Bob", 5));
    officePrinter.submitJob(Job(3, "Charlie", 24));

    // Print one job right away
    officePrinter.printNext();

    std::cout << "\n=== More Jobs Arriving ===" << std::endl;
    officePrinter.submitJob(Job(4, "Diana", 3));
    officePrinter.submitJob(Job(5, "Evan", 18));
    officePrinter.submitJob(Job(6, "Fiona", 7));

    // Try to go over capacity (limit is 5, currently holds 5)
    officePrinter.submitJob(Job(7, "George", 10));

    // Clear out remaining jobs
    officePrinter.printAll();

    return 0;
}
```

Driver Function

Output

```
Constructor class was called
=== Submitting Initial Jobs ===
[Error] Printer queue is full! Couldn't add Job #1 (Alice, 12 pages)
[Error] Printer queue is full! Couldn't add Job #2 (Bob, 5 pages)
[Error] Printer queue is full! Couldn't add Job #3 (Charlie, 24 pages)
No jobs in the queue to print.

=== More Jobs Arriving ===
[Error] Printer queue is full! Couldn't add Job #4 (Diana, 3 pages)
[Error] Printer queue is full! Couldn't add Job #5 (Evan, 18 pages)
[Error] Printer queue is full! Couldn't add Job #6 (Fiona, 7 pages)
[Error] Printer queue is full! Couldn't add Job #7 (George, 10 pages)

--- Starting printer run ---
--- Done! Queue is empty. ---
```

Analysis

This activity showed us how to build a shared printer simulation using an array-based queue. Which instructed us to first create a basic Job struct to store the user details and page counts, then

	implement a circular queue class using a dynamic array on the heap, then manage the queue pointers so items wrap around smoothly without shifting elements, and finally wrap everything inside a Printer class that controls job submissions and processing. This gave us the basic knowledge and steps in order to manage custom array-based queues so now we are able to utilize circular buffer data structures in our future coding endeavors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.